

Response to Updated Review C

Thank you for the follow-up. We revised the paper to clarify both the impossibility theorem’s scope and the theoretical contribution of IntConv.

1. Scope of the impossibility result

We agree that the previous statement did not make clear that the theorem does not establish impossibility for incrementalizations of total computable queries. It concerns arbitrary circuits when the chosen primitives make the composed DBSP language Turing-complete. We have made it clear in the footnote 9.

Precisely, there exists a fixed level-1 circuit with no computable sound and complete fixpoint detector. Such a detector must never report convergence incorrectly and must eventually report it whenever it occurs; it need not decide nonconvergence. We now call this “impossibility of exact fixpoint detection” instead of “undecidability” to avoid confusion.

Section 4.2 now gives an explicit bounded-evaluator circuit: its output is always zero for nonhalting program codes and eventually always one for halting codes. An exact detector would therefore recognize nonhalting, which is not recursively enumerable. The core reduction is formalized in Lean 4. Crucially, stream elimination is defined only for nonhalting codes, so this theorem does not apply to the total computable queries in your argument.

Your comparison strategy can yield an exact detector when equality with the reference answer guarantees a true fixpoint. Agreement *at* a fixpoint alone does not guarantee this converse: an intermediate answer may transiently equal the final answer and later change. Our theorem does not rule out detectors exploiting a terminating reference algorithm or query-specific structure.

We retain the theorem as a limit for exact fixpoint detection to motivate IntConv as a sufficient convergence criterion.

2. IntConv’s contribution and lack of evaluation

We agree that detector overhead could erase the benefit of incremental execution. We have not implemented or benchmarked the global IntConv detector and make no empirical efficiency claim.

IntConv’s contribution is theoretical: it formalizes the established internal-state-stability strategy and supplies soundness and completeness guarantees. The sound and complete StFP detector, combined with fixed-input and zero-output checks, makes IntConv detectable. Section 6 presents it as a correctness-oriented fallback; implementations can optimize it or use specialized detectors, such as FirstZero for naive and semi-naive query bodies.

Feldera already implements operator-level stability checks. Our additional capability is a general compositional treatment of nested bracketed circuits. Meaningful performance measurements require integration into a complete system: overhead depends on state representations, scheduling, workloads, and other optimizations abstracted away by our calculus. An isolated implementation would not resolve the end-to-end comparison. Sections 2 and 5 now qualify the implementation argument as qualitative, and the conclusion explicitly leaves implementation, overhead measurement, and comparison with non-incremental recomputation to future work.